HydroTwin OS

AI-Native Water Intelligence Operating System

for Critical Infrastructure & Data Centers

Full Project Design & Architecture Document

Version 1.0 · 2025

Executive Summary

HydroTwin OS is a full-stack, multi-agent AI system designed to optimize and make accountable the water and energy footprint of data centers — the fastest-growing consumers of freshwater in the digital economy. As AI compute demand doubles every 18 months, the water required to cool these facilities is reaching crisis levels in drought-prone regions globally.

Unlike conventional monitoring dashboards that show problems after they occur, HydroTwin OS is an active operating system for water — it perceives physical conditions in real time, reasons about trade-offs across the Water-Energy-Carbon nexus, takes autonomous control actions, and explains those decisions in plain language to operators and regulators.

The Core Innovation

HydroTwin OS unifies five independent intelligence planes — structural optimization, anomaly detection, carbon-aware control, regulatory compliance, and public transparency — into a single event-driven system where every plane shares data with every other. A leak detected in Plane 2 automatically updates the physics simulation in Plane 1, re-triggers the RL agent in Plane 3, adds to the audit trail retrieved by Plane 4, and surfaces in the public-facing Chrome Extension of Plane 5. This is not a collection of features — it is an integrated operating system.

Metric	What HydroTwin Targets	Industry Baseline
WUE (Water Usage Effectiveness)	< 0.5 L/kWh	1.4 – 2.0 L/kWh
PUE (Power Usage Effectiveness)	< 1.2	1.5 – 1.8

Leak Detection Latency	< 3 seconds	Hours to days (manual)
Regulatory Compliance Reports	Automated, real-time	Manual, quarterly
Carbon-Aware Decisions	Continuous, sub-minute	None or manual

Project Thesis & Problem Statement

The Water Crisis Nobody Talks About

A single hyperscale data center can consume between 1 and 5 million gallons of water per day — more than a small city. Training a large language model like GPT-3 consumed approximately 700,000 liters of freshwater. As generative AI demand accelerates, facilities are being approved and built in water-stressed regions — Phoenix, Las Vegas, Hyderabad, Chennai — where local aquifers are already under severe strain.

The problem operates at three levels simultaneously, and existing tools address only one at a time:

- **Level 1:** Operational level — Facility engineers lack real-time, AI-driven insight into the trade-offs between evaporative cooling (uses water) and electrical chillers (uses carbon-heavy grid power).
- **Level 2:** Structural level — Poor data center layouts force cooling systems to pump water further than necessary, wasting energy and increasing leak risk. These decisions are made once at construction and never revisited.
- **Level 3:** Governance level — City planners and regulators approve permits for massive facilities with almost no modeling of downstream water table impact. Corporate sustainability reports are self-reported and largely unverifiable.

Why Existing Solutions Fall Short

Current tools — DCIM platforms like Nlyte or Sunbird, or cloud provider sustainability dashboards — are retrospective. They aggregate monthly metrics and produce reports. They do not model the future, do not take autonomous action, do not cross-validate corporate claims against physical sensor data, and do not communicate with the public. HydroTwin OS addresses all five failure modes simultaneously.

System Architecture Overview

HydroTwin OS is organized around three architectural principles: a unified event mesh that every plane reads and writes, a shared knowledge graph that gives every AI agent access to the same grounded facts, and a layered intelligence stack from raw sensor data to natural language explanation.

The Unified Event Mesh

Every data flow in the system passes through Apache Kafka. This is the central nervous system. Rather than modules communicating directly with each other — which creates brittle point-to-point dependencies — every module publishes events to named Kafka topics and subscribes to the topics it needs. This architecture means:

- A structural layout change computed by Plane 1 is immediately available to the RL agent in Plane 3 without any code coupling between the two modules.
- A leak detection alert from Plane 2 is simultaneously written to the operator dashboard, the audit trail in the vector database, and the Chrome Extension's backend API.
- New regulatory documents ingested by Plane 4 automatically re-index into the vector database, enriching RAG responses without manual intervention.
- The entire system is observable: every event has a timestamp, source plane, and correlation ID, enabling full OpenTelemetry tracing from physical sensor to operator action.

Kafka Topic Map

sensor.raw → sensor.cleaned → ml.forecasts → rl.actions → anomaly.alerts → audit.log → public.api.eventslayout.updates → physics.recompute → rl.constraintsregulatory.docs → rag.index → compliance.reports

The Shared Knowledge Graph

Neo4j serves as the persistent memory of the system. It stores a graph where nodes represent physical assets (racks, pipes, pumps, sensors, building columns), events (anomaly detections, RL actions, regulatory filings), and concepts (cooling efficiency curves, carbon intensity zones). Edges represent relationships: 'Rack-37 is connected to Pipe-12,' 'Anomaly-449 caused Action-891,' 'Facility-Phoenix is in WaterStress-Zone-Extreme.'

This graph is what allows Plane 4's regulatory AI agent to answer questions like: 'Show me every near-miss cooling event in the last 18 months and which RL action was taken in response' — with actual retrieved evidence rather than hallucinated summaries.

Intelligence Stack Layers

Layer	Technology	Purpose

Ingestion	MQTT → Kafka → InfluxDB	Raw sensor streams to time-series store
Processing	Apache Flink (stream) / Spark (batch)	Cleaning, interpolation, feature engineering
Physics Twin	PyTorch PINNs + GNN	Differentiable simulation of physical layout
ML Forecasting	LSTM + Prophet	Load and ambient condition prediction
RL Control	Soft Actor-Critic (SAC)	Autonomous cooling control actions
Vision Anomaly	YOLOv8 + Multimodal Fusion Transformer	Leak and hotspot detection
Agent Orchestration	AutoGen + LangChain	Multi-agent task routing and RAG
Vector Store	Qdrant	Document embedding and semantic retrieval
Knowledge Graph	Neo4j	Relational asset and event memory
API Layer	FastAPI + GraphQL	Unified data access for all frontends
Operator Dashboard	Next.js + Recharts + Mapbox	Real-time monitoring and control UI
Public Layer	Chrome Extension + ONNX	User-facing water footprint visibility
Observability	OpenTelemetry + Grafana	End-to-end tracing and performance

Plane 1 — The Geometry Intelligence Engine

Problem

Data center layout decisions are made once, at construction, based on static CAD drawings and engineering intuition. Once racks are installed and pipes are routed, the thermal and fluid dynamics consequences of that layout are accepted as fixed costs. Poor layouts force cooling systems to pump water further, create thermal mixing zones that require overcooling entire aisles to manage one hotspot, and increase the length of pipe runs — which directly increases leak risk surface area.

What It Does

Plane 1 maintains a live 3D digital twin of the data center's physical geometry — not as a static visualization, but as a continuously updated differentiable physics simulation. The twin is a Graph Neural Network (GNN) where each node represents a physical asset (rack, column, pipe junction, cooling unit, floor tile) and each edge represents a physical relationship (thermal conductance, fluid flow path, spatial proximity).

The GNN is augmented with Physics-Informed Neural Network (PINN) constraints — the loss function includes terms derived from the Navier-Stokes equations for pipe flow and the heat equation for rack thermal propagation. This means the model cannot produce physically impossible outputs: it cannot recommend a pipe routing that violates pressure laws, or a rack arrangement that creates a thermal gradient beyond what the materials can sustain.

> **Key Technical Innovation**
> PINNs constrain the GNN to physically valid solutions. Standard ML models optimizing layout would find numerical minima that violate fluid dynamics. By embedding the governing physics equations directly into the loss function, the model learns the feasible solution space — making its recommendations directly implementable by a mechanical engineer without additional validation.

Input / Output

Input	.dxf CAD files, current sensor readings (rack power draw, coolant flow rate, temperature gradients), pipe pressure sensor streams from Kafka
Processing	GNN forward pass over the physical graph, PINN loss evaluation, gradient-based layout optimization using Adam optimizer
Output	Optimized rack arrangement recommendations, minimum-cost pipe routing paths, thermal risk heatmap, updated Kafka event for Plane 3 to re-evaluate RL constraints
Continuous Update	Re-runs optimization whenever Plane 3's RL agent takes a cooling action that changes the effective thermal load distribution

Tech Stack

- PyTorch — GNN implementation (torch_geometric library) and PINN loss functions
- ezdxf — Python library for parsing and generating .dxf CAD files
- NetworkX — Graph construction and layout algorithm preprocessing
- SciPy — Numerical solvers for fluid dynamics validation
- Apache Kafka — Publishes layout update events, subscribes to sensor streams
- Neo4j — Stores the asset graph persistently with relationship metadata
- Three.js — 3D visualization of the twin in the Next.js dashboard

Data Sources (All Free)

- ASHRAE TC 9.9 thermal guidelines — Published rack temperature operating envelopes
- Procedurally generated synthetic floor plans — Python script produces randomized but physically valid layouts using standard data center geometry rules
- Live sensor streams — Simulated from InfluxDB using published Google/Meta data center efficiency parameter ranges

Plane 2 — The Multimodal Anomaly Cortex

Problem

As data centers adopt closed-loop liquid cooling at the rack level, the failure mode shifts from 'too hot' to 'microscopic leak.' A pinhole leak in a high-pressure coolant line can destroy millions of dollars of hardware within minutes and goes completely undetected by temperature sensors until hardware begins failing. Existing solutions rely on floor moisture sensors — which only detect leaks that have already reached the floor — or quarterly visual inspections.

What It Does

Plane 2 is a continuous real-time anomaly detection pipeline that fuses four independent sensor modalities to detect failures with far higher confidence than any single sensor. The key insight is that different failure modes produce different multi-modal signatures: a pump failure shows as a thermal hotspot AND a loss of flow rate AND a change in vibration frequency. A coolant leak shows as a thermal gradient change AND a moisture sensor spike AND (if advanced) a chemical composition change in nearby air sensors.

A late-fusion multimodal transformer processes all four streams simultaneously. Each modality has its own encoder — YOLOv8 for thermal/visual imagery, a 1D CNN for vibration time series, an LSTM for chemical sensor streams, and a threshold-gated alert for moisture point sensors. The transformer's cross-attention layers learn which modality combinations are most diagnostic for each failure type.

> **Why Multi-Modal Fusion Matters**
>
> Single-modality systems have high false-positive rates. A thermal hotspot could be a dense AI workload or a failing pump — temperature alone cannot distinguish them. When thermal anomaly co-occurs with a specific vibration frequency signature (pump cavitation) and no corresponding workload spike in the IT metrics, the fusion model classifies this as a pump failure with high confidence. This dramatically reduces alert fatigue — the enemy of any monitoring system.

Detection Pipeline

Camera Feed	YOLOv8 processes simulated thermal IR frames at 10fps. Detects bounding boxes around temperature anomalies. Outputs: coordinates, confidence, severity class.
Vibration Sensors	1D CNN processes 1kHz vibration time series from simulated accelerometers on pump housings. Classifies cavitation, bearing wear, or normal operation.
Chemical Sensors	LSTM processes VOC (Volatile Organic Compound) sensor streams. Coolant evaporation produces a characteristic chemical signature before visible condensation forms.

Moisture Grid	Point moisture sensors on server room floor tiles produce binary alert signals. Threshold-gated, fed into fusion layer as a hard constraint.
Fusion Layer	Multimodal transformer combines all four streams. Outputs: event type, location coordinates, confidence score, recommended action.
Output Actions	Writes to Kafka anomaly.alerts topic. Simultaneously: operator dashboard push alert, Neo4j event node creation, audit trail entry for Plane 4 RAG, Chrome Extension notification.

Tech Stack

- YOLOv8 (Ultralytics) — Object detection on thermal frames, fine-tuned on FLIR thermal dataset
- PyTorch — Multimodal fusion transformer, 1D CNN, LSTM encoders
- OpenCV — Frame preprocessing and annotation
- Apache Kafka — Real-time stream ingestion and alert publishing
- InfluxDB — Time-series storage for all sensor modalities
- FastAPI — Anomaly alert webhook endpoint
- Neo4j — Anomaly event nodes linked to affected physical asset nodes

Data Sources (All Free)

- FLIR ADAS Thermal Dataset — Open thermal imaging dataset for fine-tuning YOLOv8
- Simulated sensor streams — Python generators producing physically plausible sensor data based on published failure mode signatures from ASHRAE and IEEE data center reliability papers

Plane 3 — The Water-Energy-Carbon Nexus Agent

Problem

The naive approach to water conservation — 'use less water' — fails because cooling is a physical constraint, not an option. When you reduce evaporative cooling, you must compensate with electric chillers. If the local power grid is coal-heavy, that trade-off increases carbon emissions. If there is a severe heat wave, outside air economization (free cooling using ambient air) becomes ineffective. The system must continuously solve a multi-objective problem with shifting constraints and no single right answer.

What It Does

Plane 3 is a Reinforcement Learning controller that has learned to navigate the Water-Energy-Carbon Pareto frontier — the set of operating points where you cannot improve one metric without worsening another. The controller observes the current state of the facility, the power grid's marginal carbon intensity, weather conditions, and IT workload forecasts, and selects cooling actions that land on the optimal point of the Pareto frontier given the operator's current priority setting.

The RL agent uses Soft Actor-Critic (SAC) — an off-policy algorithm that optimizes a maximum-entropy objective, meaning it learns to be effective while also being uncertain. This prevents the agent from becoming overconfident in edge cases it has rarely seen. The agent was trained entirely in simulation using the Plane 1 physics twin as its environment — it has never needed access to a real data center.

> **The Reward Function — The Core Research Contribution**
>
> $R(t) = -\alpha \cdot WUE(t) - \beta \cdot PUE(t) - \gamma \cdot CarbonIntensity(t) + \delta \cdot ThermalConstraintSatisfaction(t)$ α, β, γ are operator-configured weights that shift the agent along the Pareto frontier. Setting α=1, β=0, γ=0 minimizes water at any energy cost. Setting γ=1, α=0, β=0 minimizes carbon. In practice, weights are dynamically adjusted based on real-time grid carbon data from ElectricityMaps — automatically shifting toward water use when the grid is clean, and toward chillers when the grid is renewable-heavy.

The Workload Migration Subsystem

When local conditions become extreme — severe heat wave making outside air economization impossible, combined with a dirty grid and a water stress alert — the agent triggers a workload migration recommendation. This queries a live multi-region carbon intensity map and identifies the geographic region with the lowest marginal emissions per unit of compute. It then interfaces with the Kubernetes Cluster Federation API to dispatch deferrable workloads (batch ML training jobs, video transcoding, cold storage replication) to that region.

Tech Stack

- Stable-Baselines3 — SAC implementation, training loop, and evaluation utilities
- PyTorch — Custom neural network architectures for actor and critic networks
- Custom Gym Environment — Built on the Plane 1 physics twin, simulates data center thermal response to RL actions
- ElectricityMaps API (free tier) — Real-time and forecast grid carbon intensity per region
- NOAA Climate Data Online API (free) — Hourly dry-bulb and wet-bulb temperature, relative humidity
- WRI Aqueduct API (free) — Water stress index per geographic watershed
- Prophet (Meta) — IT load forecasting to give RL agent a planning horizon
- PyTorch LSTM — Alternative load forecasting model for ensemble
- Kubernetes Cluster API — Programmatic workload dispatch to alternate regions
- Apache Kafka — Publishes RL actions to rl.actions topic, consumed by Plane 1 and dashboard

Plane 4 — The Regulatory Intelligence & Siting Brain

Problem

City planners and water regulators approve construction permits for hyperscale data centers with almost no quantitative modeling of the facility's downstream effect on local water tables, watershed health, or grid stress. Corporate ESG sustainability reports are self-reported, unaudited, and standardized in ways that make greenwashing trivially easy — a company can report a WUE improvement while simultaneously tripling the absolute volume of water consumed by building more facilities.

What It Does

Plane 4 is a multi-agent deliberative system that allows regulators and city planners to ask natural language questions and receive structured, evidence-backed answers grounded in retrieved documents and real operational data. It is powered by a RAG (Retrieval-Augmented Generation) pipeline backed by a Qdrant vector database that indexes EPA regulations, ASHRAE guidelines, WRI watershed reports, GRACE satellite groundwater data, and — critically — the operational event logs produced by Planes 1, 2, and 3 of the system itself.

The deliberation pattern uses AutoGen's group chat architecture with three specialized agents debating incoming questions:

- Water Scientist Agent — Retrieves and reasons over hydrological documents. Specializes in watershed stress modeling and groundwater depletion projections.
- Grid Stress Agent — Retrieves FERC utility capacity filings and real-time grid load data. Assesses whether a proposed facility's power demand is consistent with the local grid's renewable capacity roadmap.
- Adversarial Auditor Agent — Cross-references a company's public sustainability claims against the operational data ingested from Planes 1-3. Flags discrepancies between reported WUE and sensor-derived WUE estimates. Assigns a greenwashing risk score.

The Geographic Stress Test Engine

A parameterized Monte Carlo simulation engine allows any city, any facility size, and any climate scenario to be stress-tested before construction approval. The Hyderabad, Telangana case study — simulating a 50MW facility against the region's seasonal water table and peak summer wet-bulb temperatures — is one instance of a generalized engine.

Inputs to the simulation include GRACE-FO satellite groundwater anomaly data (free via NASA), seasonal precipitation forecasts from NOAA, current water stress classification from WRI Aqueduct, and the proposed facility's power density and cooling technology specification. The engine runs 10,000 Monte Carlo samples across a 20-year projection window and outputs a probability distribution over water table depletion scenarios, along with the 5th percentile worst case that regulatory approval should be conditioned upon.

Tech Stack

- AutoGen — Multi-agent group chat orchestration (Water Scientist, Grid Stress, Auditor agents)
- LangChain — RAG pipeline, document loading, text splitter, retrieval chain
- Qdrant (local or cloud free tier) — Vector database for document embeddings
- Google Gemini API (free tier) — LLM backbone for all three agents
- sentence-transformers — Document embedding model (runs locally, no API cost)
- NASA GRACE-FO API (free) — Satellite groundwater anomaly data
- WRI Aqueduct API (free) — Water stress index and seasonal water availability
- NOAA CDO API (free) — Climate normals and seasonal precipitation data
- NumPy / SciPy — Monte Carlo simulation engine for stress testing
- Streamlit — Regulatory dashboard UI (separate from main Next.js dashboard, optimized for non-technical regulatory users)
- ReportLab — Automated PDF compliance report generation

Plane 5 — The Public Accountability Layer

Problem

The public has no visceral sense of the physical resource cost of their digital actions. When someone asks an AI chatbot a question, they see a text response. They do not see the gallons of water evaporated in a data center cooling tower, the kilowatt-hours drawn from the grid, or the grams of CO2 emitted by the marginal generator that turned on to serve that request. Without this visibility, there is no public pressure for corporate accountability, and no personal agency over one's own digital carbon and water footprint.

What It Does

A Chrome Extension that runs entirely client-side — no user data leaves the browser — intercepts network requests and classifies them by compute intensity using a tiny ONNX model (under 5MB) that runs in the browser's WASM runtime. The classification model was trained to distinguish between lightweight requests (loading a static webpage) and heavy compute requests (generative AI queries, video transcoding, large-scale data retrieval).

When a heavy request is classified, the Extension queries the HydroTwin public API — which aggregates facility-level WUE data from public ESG disclosures and voluntary facility operator contributions — to retrieve the estimated WUE and grid carbon intensity for the identified cloud provider's relevant region. It then calculates and displays an estimate: '~18ml of freshwater, ~4.2g CO2' in a clean, unobtrusive popup. Over a session, it accumulates totals and provides weekly summaries.

> **Privacy First**
> The ONNX classification model runs entirely in the browser using WebAssembly. No browsing data, URLs, or request content is sent to any server. The only external call is a lookup to the HydroTwin public API using a provider identifier (e.g., 'openai-us-east') — not the user's query. This design is intentional and auditable.

Tech Stack

- Chrome Extension Manifest V3 — Service worker architecture for request interception
- ONNX Runtime Web — In-browser inference of the request classification model
- Transformers.js / custom ONNX model — Lightweight request classifier trained on HTTP traffic patterns
- FastAPI — Public API backend serving aggregated WUE and carbon intensity data
- React — Extension popup UI
- Web Workers — Ensures ONNX inference does not block the browser UI thread

Complete Technology Stack Reference

Data Infrastructure

Component	Technology	Purpose	Cost
Event Streaming	Apache Kafka (Confluent Cloud)	Central nervous system — all inter-plane communication	Free tier
Time-Series DB	InfluxDB Cloud	All sensor metric storage	Free tier
Knowledge Graph	Neo4j AuraDB	Asset and event relational memory	Free tier (1GB)
Vector Store	Qdrant Cloud	Document embeddings for RAG	Free tier
Stream Processing	Apache Flink (local)	Real-time sensor data cleaning	Free (open source)
Object Storage	Google Drive / local	Model checkpoints, raw data	Free

Machine Learning & AI

Component	Technology	Purpose	Cost
GNN + PINN	PyTorch + torch_geometric	Physics-constrained layout optimization	Free
RL Agent	Stable-Baselines3 (SAC)	Multi-objective cooling control	Free
Object Detection	YOLOv8 (Ultralytics)	Thermal and visual anomaly detection	Free
Fusion Transformer	PyTorch (custom)	Multi-modal sensor fusion	Free
Load Forecasting	Prophet + LSTM	IT load and ambient condition prediction	Free
LLM Backend	Google Gemini API	RAG agents, natural language interface	Free tier
Embeddings	sentence-transformers	Document vectorization for Qdrant	Free (local)
Browser ML	ONNX Runtime Web	In-browser request classification	Free
Training Compute	Google Colab (T4 GPU)	Model training	Free

Agent Orchestration

Component	Technology	Purpose	Cost

Multi-Agent Framework	AutoGen (Microsoft)	Group chat deliberation for regulatory agents	Free
RAG Orchestration	LangChain	Document loading, retrieval chain, prompt management	Free
Agent Memory	Neo4j + Qdrant	Persistent grounded memory across sessions	Free

External APIs (All Free Tier)

API	Provider	Data Provided	Rate Limit
Grid Carbon Intensity	ElectricityMaps	Real-time and forecast CO2 per kWh per region	Free developer tier
Groundwater Anomaly	NASA GRACE-FO	Satellite-derived water table changes	No limit (bulk download)
Water Stress Index	WRI Aqueduct	Watershed stress classification globally	Free API
Weather & Climate	NOAA CDO	Temperature, humidity, precipitation normals	Free, 1000 req/day
CAD Parsing	ezdxf (local)	Parse .dxf CAD files	No limit (local library)

Application Layer

Component	Technology	Purpose	Cost
Operator Dashboard	Next.js 14 + Tailwind CSS	Real-time monitoring, RL control panel, 3D twin view	Free (Vercel hosting)
Charts & Visualization	Recharts + D3.js + Three.js	Metrics, forecasts, 3D facility rendering	Free
Map Visualization	Mapbox GL JS	Geographic stress test visualization	Free tier
Regulatory Dashboard	Streamlit	Non-technical regulator interface	Free (Streamlit Cloud)
Backend API	FastAPI + GraphQL (Strawberry)	Unified data access layer	Free (Railway/Render)
Report Generation	ReportLab	PDF compliance report output	Free
Chrome Extension	Manifest V3 + React	Public water footprint visibility	Free

DevOps & Observability

Component	Technology	Purpose	Cost
Containerization	Docker + Docker Compose	Local development environment	Free
Orchestration	Minikube (local Kubernetes)	Production-equivalent container orchestration	Free
CI/CD	GitHub Actions	Automated testing and deployment pipelines	Free
Tracing	OpenTelemetry + Jaeger	End-to-end request tracing across all planes	Free
Metrics	Prometheus + Grafana	System health and performance dashboards	Free
Version Control	Git + GitHub	Source control, project documentation	Free

End-to-End Data Flow

Scenario: Leak Detection to Operator Action

The following describes the complete data path for a cooling system anomaly event, illustrating how all five planes and the shared infrastructure interact:

- **Step 1:** T+0.0s — Simulated thermal camera frame shows a temperature gradient deviation of 4°C above baseline in Aisle 7, Rack 23. YOLOv8 detects bounding box with 0.87 confidence. Frame annotated and published to Kafka topic: sensor.vision.thermal.
- **Step 2:** T+0.1s — Vibration LSTM simultaneously detects cavitation frequency signature in pump P-07 (which serves Aisle 7). Published to Kafka topic: sensor.vibration.
- **Step 3:** T+0.2s — Multimodal fusion transformer receives both events via Kafka consumer. Cross-attention layer identifies thermal + vibration co-occurrence. Classifies event as: PUMP_FAILURE, confidence 0.94, location (Aisle 7, Rack 23, Pump P-07).
- **Step 4:** T+0.3s — Anomaly event published to Kafka topic: anomaly.alerts with full metadata. Simultaneously consumed by: (a) Operator Dashboard WebSocket for push notification, (b) Neo4j writer creating event node linked to Pump P-07 asset node, (c) Plane 4 RAG indexer adding event to audit trail vector index, (d) Chrome Extension public API for subscriber notification.
- **Step 5:** T+0.5s — Plane 3 RL agent receives anomaly event. Pump P-07 failure means cooling capacity in Aisle 7 is reduced. Agent recomputes action: increase chiller output to compensate, reduce workload scheduled to Aisle 7 racks, publish new cooling setpoints to rl.actions Kafka topic.
- **Step 6:** T+0.6s — Plane 1 GNN receives rl.actions event indicating changed thermal load distribution. Recomputes thermal propagation across physical graph. Identifies that the new distribution increases thermal stress on adjacent Aisle 8. Publishes layout risk update.
- **Step 7:** T+1.0s — Operator receives dashboard alert with: anomaly type, confidence, affected equipment, automated RL response taken, and adjacent risk flag from Plane 1. Total latency from camera frame to full operator situational awareness: ~1 second.
- **Future:** T+later — Regulatory agent in Plane 4 can now respond to the query: 'How many pump failures has Facility X had in the last 6 months and what was the automated response each time?' with retrieved, grounded evidence from the Neo4j and Qdrant audit trail.

Build Roadmap

The recommended build sequence prioritizes early demonstrability. Each phase produces an independently showable artifact, meaning you can demonstrate progress to recruiters or collaborators at any stage without waiting for the full system to be complete.

Phase	Duration	Deliverable	Planes Involved
Phase 0: Foundation	Week 1-2	Kafka + InfluxDB running locally. Python sensor simulator publishing synthetic data. InfluxDB dashboard showing live streams.	All (shared infra)
Phase 1: Carbon Nexus Agent	Week 3-6	SAC RL agent trained in simple thermal simulation. Connected to ElectricityMaps and NOAA APIs. Logs optimal cooling setpoints to dashboard.	Plane 3
Phase 2: RAG Regulatory Brain	Week 7-9	LangChain RAG pipeline with EPA + ASHRAE documents in Qdrant. AutoGen 3-agent deliberation working. Streamlit dashboard live.	Plane 4
Phase 3: Anomaly Cortex	Week 10-13	YOLOv8 fine-tuned on FLIR thermal data. Multimodal fusion transformer prototype. End-to-end detection pipeline with Kafka.	Plane 2
Phase 4: Chrome Extension	Week 14-15	Browser extension classifying requests. Public API serving WUE estimates. Live and publishable.	Plane 5
Phase 5: Physics Twin	Week 16-21	GNN + PINN on synthetic floor plan graphs. Integrated with Plane 3 RL agent as training environment. 3D visualization in dashboard.	Plane 1
Phase 6: Integration	Week 22-26	Unified Kafka event mesh connecting all planes. OpenTelemetry tracing. Full operator dashboard. End-to-end demo scenario.	All planes

Running the Entire System for Free

Compute Strategy

All model training happens on Google Colab's free T4 GPU. The key discipline is saving checkpoints to Google Drive after every training epoch — Colab sessions disconnect and you must be able to resume without losing progress. For the SAC RL agent training (the most compute-intensive component), a typical training run of 500,000 environment steps takes approximately 4-6 hours on a T4, which fits within Colab's free session window with checkpointing.

Service Hosting Strategy

Every persistent service runs on a free tier during development and portfolio demonstration. The FastAPI backend deploys to Railway or Render (both free for low-traffic apps). The Next.js dashboard deploys to Vercel (free). The Streamlit regulatory dashboard deploys to Streamlit Community Cloud (free). InfluxDB, Qdrant, and Neo4j all have free cloud tiers sufficient for a demonstration system. Kafka runs on Confluent Cloud's free tier (5GB storage, sufficient for a demo).

LLM API Cost

The Google Gemini API provides 15 requests per minute and 1 million tokens per day on the free tier — more than sufficient for a RAG system that processes human queries. Gemini 1.5 Flash (the free tier model) is capable enough for the regulatory question-answering task. The sentence-transformers embedding model runs locally with no API calls required, eliminating embedding costs entirely.

> **Total Estimated Cost to Build Complete System**
>
> Free tier cloud services: $0. Google Colab GPU training: $0 (free tier sufficient). External APIs: $0 (all have adequate free tiers). Domain name (optional): ~$10/year. Total: $0 to build. Optional $10 for a custom domain to make your demo URL professional.

Glossary of Key Terms

WUE	Water Usage Effectiveness — liters of water consumed per kilowatt-hour of IT equipment energy. Industry average: 1.4 L/kWh. World-class: < 0.5 L/kWh.
PUE	Power Usage Effectiveness — ratio of total facility energy to IT equipment energy. A PUE of 1.0 is perfect (all power goes to compute). Industry average: 1.5-1.8.
PINN	Physics-Informed Neural Network — a neural network whose loss function includes terms derived from physical governing equations (e.g., Navier-Stokes, heat equation), ensuring outputs respect physical laws.
GNN	Graph Neural Network — a neural network that operates on graph-structured data, where nodes and edges carry feature vectors and message-passing updates node representations based on neighborhood structure.
SAC	Soft Actor-Critic — an off-policy reinforcement learning algorithm that maximizes a trade-off between expected reward and policy entropy, producing robust agents that handle uncertainty well.
RAG	Retrieval-Augmented Generation — a technique where an LLM's responses are grounded by first retrieving relevant documents from a vector database, preventing hallucination and enabling citation.
Pareto Frontier	In multi-objective optimization, the set of solutions where you cannot improve one objective without worsening another. The Water-Energy-Carbon nexus has a Pareto frontier the RL agent learns to navigate.
Kafka Topic	A named, partitioned, ordered log of events in Apache Kafka. Publishers write events to topics; subscribers read from them. Topics are the communication channels of the unified event mesh.
GRACE-FO	Gravity Recovery and Climate Experiment Follow-On — a pair of NASA satellites that measure changes in Earth's gravitational field caused by water mass redistribution, enabling groundwater monitoring.
Wet-Bulb Temperature	The lowest temperature achievable by evaporating water into air. Critical for data center cooling: when wet-bulb temperature exceeds ~35°C, evaporative cooling becomes ineffective and chillers must take over.

Why This Project Cannot Be Ignored

Every technical role you will interview for has a version of this project that speaks directly to its core concerns. The project is not a single trick — it is a full-stack demonstration that you can operate at every level of the technical stack simultaneously, from the governing equations of fluid dynamics to a user-facing Chrome Extension popup.

For ML and AI research roles, the PINN-constrained GNN and the multi-objective SAC agent are credible independent implementations of frontier research problems that Google DeepMind, Microsoft Research, and national labs are actively working on. For systems engineering roles, the Kafka event mesh with OpenTelemetry end-to-end tracing demonstrates distributed systems design at production scale. For applied AI and product roles, the RAG deliberation system and the Chrome Extension demonstrate the complete arc from model to user.

Most importantly, it is about something real. Water scarcity is not a hypothetical future problem — it is already causing data center construction moratoriums in Amsterdam, Dublin, and Singapore. This project provides tools that a regulator, a facility engineer, or a concerned citizen could actually use today. That combination of technical depth and genuine real-world relevance is what makes it impossible to dismiss, regardless of the seniority of the role.

— End of Document —